CSA0506 – Database Management Systems CO4 ASSESSMENT TOOL 3 – Industry Problem Solving Task	Name: Dharanitharan G
	Reg: 192511396
	B.E - CSE

Q1. An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.

An e-commerce catalog is one of the most heavily read data stores in the entire platform: every product page view, every search result, every "add to cart" action, and every price check triggers a lookup against the product record. Understanding the dominant access pattern is therefore the starting point for choosing a file organization, because different organizations are optimized for very different workloads. Sequential/heap files are optimized for bulk scans, sorted files are optimized for ordered range access, and hashed files are optimized for exact-match point lookups. Since the overwhelming majority of e-commerce queries are of the form "fetch the record for Product_ID = X", a file organization built around fast equality lookup is the correct starting assumption.

Recommended Strategy: Hashed File Organization

A hashed file organization is recommended, ideally implemented as Extendible Hashing rather than plain static hashing, because a catalog of 10 million products is large enough that the bucket directory will need to grow over time as new products are added, seasonal catalogs expand, or the company acquires new product lines. Extendible hashing allows the directory to double incrementally instead of requiring the entire 10-million-record file to be rehashed whenever the load factor becomes too high, which would otherwise require taking the catalog offline or running an expensive background reorganization job.

Cost Analysis

Assume each product record is approximately 50 bytes and the block size is 4KB, giving a blocking factor of roughly 80 records per block, which means the 10 million records occupy approximately 125,000 blocks of storage. The following table compares the expected I/O cost of the three classical file organizations for this dataset:

Organization	Search Cost (I/Os)	Insert Cost	Space Overhead
Heap File	O(n) ≈ 125,000 (average n/2 ≈ 62,500)	O(1) — append only	None
Sorted File	$O(\log_2 n) \approx 17$	O(n) — requires shifting / reorganization	None (but low fill-factor in practice)
Hash File (Extendible)	O(1) ≈ 1–2	O(1) amortized, occasional directory doubling	20–30% for buckets and directory overhead

The 20–30% extra storage that hashing requires (roughly 25,000 to 37,500 additional blocks, translating to a modest amount of extra disk space at commodity cloud storage prices) is a negligible monetary cost when weighed against the operational savings. If the platform serves even a few million lookups per day, the difference between an average of 62,500 block accesses (heap file) and 1–2 block accesses (hash file) per lookup is the difference between an unusable system and a responsive one. Even compared to the sorted file's 17 I/Os, the hashed file's 1–2 I/Os represents an order-of-magnitude improvement, which directly translates into lower compute cost per request, lower database server sizing requirements, and better customer-facing page-load

Conclusion

Given that product catalog access is dominated by exact-match retrieval rather than ordered range scanning, and given the scale of 10 million records, Extendible Hashing is recommended. It provides near-constant-time lookups, supports graceful incremental growth of the catalog without a full file reorganization, and its modest storage overhead is easily justified by the substantial reduction in I/O cost and improvement in response time at this scale.

Q2. A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.

Banking transaction systems have a dual access requirement that makes single-index solutions inadequate: customers and tellers need to pull up an individual account's full transaction history (an equality/range lookup on Account_Number), and the same system must also support date-bounded queries such as generating a monthly statement, running fraud-detection scans over a time window, or producing regulatory reports for a specific date range. Because a single physical ordering of the file can only optimize for one of these two attributes at a time, a layered indexing strategy that combines a clustering index with one or more secondary indexes is required.

1. Primary (Clustering) Index — B+ Tree on Account Number

Because most banking systems physically group a customer's transactions together for statement generation and auditing purposes, the transaction file itself should be physically clustered and sorted by Account_Number. A B+ Tree clustering index is built on top of this physical ordering. This index allows the system to seek directly to the first block belonging to a given account and then sequentially scan forward through that account's transactions, which is the single most frequent operation in a banking system — "show me this account's activity."

2. Secondary Index — B+ Tree on Transaction Date

Since the file is physically ordered by account number, it is not simultaneously ordered by date. To support queries that need to scan transactions purely by date, irrespective of account (for example, a bank-wide report of all transactions processed on a specific business day, or a fraud-detection sweep across all accounts within a suspicious time window), a secondary, non-clustering B+ Tree index is built on Transaction_Date. Because this index is not the physical ordering of the file, its leaf nodes store pointers to the actual data blocks rather than the data itself.

3. Composite Index — B+ Tree on (Account_Number, Transaction_Date)

The most common real-world banking query combines both predicates at once — for example, "retrieve all transactions for Account 10293 between 1 January and 31 March." To serve this efficiently without first collecting all of an account's transactions and then filtering by date in application code, a composite (concatenated) B+ Tree index on (Account_Number, Transaction_Date) is built. The tree first narrows the search to the target account using the leading key, and because entries for that account are then sorted by date at the leaf level, a simple range scan across the linked leaf nodes retrieves exactly the requested window with minimal wasted I/O.

Why B+ Tree Rather Than B-Tree or a Pure Hash Index

A hash index would be excellent for exact-match lookups on Account_Number alone, but hashing fundamentally cannot support range queries because hash functions deliberately scatter keys and destroy any notion of order — a date range query against a hashed structure would degrade to a near-complete scan of the index. B-Trees, unlike B+ Trees, store data pointers in internal nodes as well as leaf nodes; this increases the cost of splits and merges during insertion and deletion, and, more importantly, does not provide a linked sequence of leaves for fast ordered scanning. B+ Trees solve both problems: they keep the tree shallow (high fan-out, since internal nodes hold only keys) and they chain all leaf nodes together in sorted order, which makes range scans — the exact requirement for date-bounded queries — very efficient.

Conclusion

A layered indexing strategy consisting of a clustering B+ Tree on Account_Number, a composite B+ Tree on (Account_Number, Transaction_Date) for the most common combined query, and a secondary B+ Tree on Transaction_Date alone for bank-wide date queries, together provide fast retrieval for every realistic access

Q3. A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.

A hospital or clinic information system is queried in two structurally different ways. Clinical staff and billing systems most often retrieve a specific patient's chart directly by PatientID, an operation that should be as close to instantaneous as possible, particularly in emergency settings. Separately, a doctor or administrator frequently needs to pull up the full list of patients under a specific physician's care, which is a query against a non-key, highly duplicated attribute (many patients share the same doctor). Because the volume of patient records in a large hospital network can run into the millions, a single-level index is not sufficient, since even the index itself would be too large to search in one or two I/Os — this is exactly the situation multi-level indexing is designed to address.

1. Primary Index — Multi-Level Clustering B+ Tree on PatientID

The patient file is physically stored in PatientID order, and a multi-level B+ Tree is built on top of it. Rather than a flat single-level index (which for millions of patients would itself span many blocks and therefore require a linear or binary scan to search), the index is organized hierarchically: a root level with a small number of entries, one or more intermediate levels, and a leaf level whose entries point to the actual data blocks. Because each level of the tree has a high fan-out, the number of blocks that must be read to locate any given patient is reduced to the height of the tree — typically only two or three I/Os even for a multi-million-record hospital network — rather than the many I/Os a single flat index would require.

2. Secondary Index — Multi-Level B+ Tree on Doctor Name with Bucketed Pointers

Since the file is not physically ordered by doctor name, a non-clustering secondary index is required. Because a doctor's name is a non-key attribute — many patients map to the same doctor — a naive secondary index that stored one entry per patient per doctor would contain a large number of duplicate keys. Instead, the recommended design uses an intermediate bucket layer: the B+ Tree index on Doctor Name points not directly to patient records but to a bucket containing the list of PatientID pointers associated with that doctor. This two-level indirection (index → bucket → record pointers) avoids duplicating the doctor's name across many index entries and keeps the secondary index itself compact enough to remain multi-level and efficient.

Access Flow Summary

- Query by PatientID: traverse the multi-level primary B+ Tree directly to the data block — fastest possible path, ideal for point-of-care lookups.
- Query by Doctor Name: traverse the multi-level secondary B+ Tree to the doctor's bucket, retrieve the list of PatientID pointers in that bucket, then use the primary index (or the pointers directly) to fetch each patient's full record.

Conclusion

A multi-level clustering B+ Tree on PatientID, paired with a multi-level non-clustering secondary index on Doctor Name that resolves through an intermediate bucket of PatientID pointers, gives the healthcare system fast, scalable access along both required dimensions, while keeping index maintenance manageable as the number of patients and physicians continues to grow.

Q4. A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.

A logistics company's shipment-tracking database is one of the most write-intensive workloads in enterprise data management: new shipments are created continuously, existing shipment statuses are updated as parcels move through the network, and completed or cancelled shipments are eventually deleted or archived. This means the indexing structure chosen must not only support fast lookups but must also tolerate a very high rate of structural change — insertions and deletions — without frequent, expensive rebalancing. This makes the comparison between B-Tree and B+ Tree indexing structures directly relevant, since the two structures behave very differently under heavy insert/delete load.

Structural Comparison

Criterion	B-Tree	B+ Tree
Data storage location	Keys and data stored at internal nodes AND leaf nodes	Data stored only at leaf nodes; internal nodes hold routing keys only
Insert/Delete rebalancing	Can require restructuring at internal levels since data pointers live there too	Restructuring is largely confined to the leaf level, simplifying splits and merges
Sequential/range access	No linked leaves — range scans require repeated tree traversal	Leaf nodes are linked in a sorted chain — ideal for range and sequential access
Fan-out per node	Lower, because internal nodes must also store data pointers	Higher, because internal nodes store only keys — shorter, shallower tree
Redundancy	No duplicate keys between levels	Keys are duplicated at the leaf level, but this simplifies maintenance and scanning

Why B+ Tree Is More Suitable for Shipment Records

First, and most importantly for this scenario, frequent dynamic insertions and deletions are handled more cheaply in a B+ Tree because restructuring is confined mostly to the leaf level. In a B-Tree, because data pointers can live in internal nodes, an insertion or deletion that triggers a split or merge may need to relocate data pointers embedded partway up the tree, which is a more expensive and more disruptive operation than simply adjusting leaf-level entries and, at most, updating a routing key in a parent node.

Second, because B+ Tree internal nodes store only keys rather than full data pointers, more keys fit into a single node of the same block size, which increases the tree's fan-out and reduces its overall height. A shallower tree means fewer nodes need to be touched — and therefore fewer nodes need to be rebalanced — for every insert or delete operation, which directly reduces the average cost of the shipment system's constant stream of writes.

Third, logistics operations are not purely about single-record lookups; they routinely require range-based operations such as "list all shipments dispatched this week" or "track all shipments between consignment ID 4000 and 4500." The linked-leaf structure of the B+ Tree directly supports these range scans by allowing the system to locate the starting key once and then simply follow leaf pointers, whereas a B-Tree would need to perform an in-order traversal that repeatedly revisits internal nodes, making it comparatively inefficient for this common operational need.

Conclusion

The B+ Tree is the more suitable structure for the logistics company's shipment records. Its leaf-only data storage keeps insertions and deletions cheap and localized, its higher fan-out keeps the tree shallow even as the volume of shipment records grows, and its linked-leaf design efficiently supports the frequent range-based tracking and reporting queries that are central to logistics operations — all of which B-Trees handle less efficiently.

Q5. Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.

An HR system's employee database is typically queried by Employee_ID for the vast majority of operations — payroll processing, benefits lookup, attendance verification, and access-control checks all begin with a single, direct lookup of one employee's record. This equality-dominated access pattern makes hashing a natural fit as the primary access method, but the specific choice between static and extendible (or dynamic) hashing depends heavily on how the organization is expected to grow over the life of the system, which is exactly what this scenario asks us to evaluate.

Proposed Hashing Scheme

The hash key chosen is Employee_ID, since it is unique, fixed-length, and — if the organization assigns IDs sequentially or through a well-distributed coding scheme — produces a reasonably uniform spread of hash values. The hash function is defined as h(Employee_ID) = Employee_ID mod M, where M is selected as a prime number close to, but somewhat larger than, the number of buckets strictly required, in order to reduce clustering. Assuming each bucket can comfortably hold about 20 employee records per 4KB block (at roughly 200 bytes per record) and allowing a buffer so buckets are not filled to capacity from day one, the required number of buckets is approximately $5000 \div 20 \times 1.25 \approx 313$, so the nearest prime, M = 313, is chosen as the modulus. Overflow chaining is used to gracefully absorb any bucket that receives more than its capacity due to hash collisions, rather than allowing the scheme to fail outright.

Static Hashing — Evaluation

Static hashing is straightforward to implement and, for a fixed and precisely known record count such as 5000 employees, offers excellent average-case O(1) access with minimal implementation complexity. Its principal weakness, however, is inflexibility: the number of buckets M is fixed at design time, so if the company's headcount grows — through organic hiring, seasonal staffing, or a merger or acquisition — the fixed bucket count leads to increasingly long overflow chains as more records collide into the same buckets, and lookup performance gradually degrades from O(1) toward something closer to a linear scan of the overflow chain. Recovering from this degradation requires choosing a new, larger M and rehashing the entire employee file, which is an expensive, disruptive, all-at-once operation that typically requires scheduled downtime or a carefully managed background migration.

Extendible Hashing — Evaluation

Extendible hashing addresses this inflexibility directly by introducing a directory of pointers to buckets, where the directory can double in size incrementally whenever a bucket overflows, rather than requiring the entire file to be rehashed at once. This allows the HR system to grow gradually and organically as headcount increases, with only the affected portion of the directory and buckets being restructured at any given time. The trade-off is a small amount of added complexity — an extra level of indirection through the directory, and occasional overhead when the directory itself needs to double — but this cost is modest and is paid incrementally rather than as one large, disruptive event.

Recommendation for This Scenario

Because 5000 employees represents a relatively stable, bounded figure typical of a mid-sized organization, static hashing is an acceptable and simpler choice if the company does not anticipate significant headcount growth in the near term. However, HR systems are almost always expected to remain in service for many years, over which time organizational growth, restructuring, or acquisitions are common. For this reason, extendible hashing is the recommended long-term design, since it avoids the periodic, disruptive full rehashing that static hashing would eventually require and instead accommodates growth smoothly as it occurs.

Conclusion

A hash scheme keyed on Employee_ID with M = 313 buckets and overflow chaining provides fast O(1) average access for this HR system. Static hashing is sufficient only if the 5000-employee count is expected to remain fixed; extendible hashing is the more robust and future-proof choice, and is recommended given the typical multi-year lifespan and organizational growth patterns of HR systems.

Q6. An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.

Airline reservation systems face two access patterns that pull in different directions. A very large share of requests are point queries — a customer or agent looking up a specific flight, such as "show me flight AI202" — which benefit from the fastest possible exact-match retrieval. At the same time, scheduling, planning, and customer-facing search tools ("show me all AI202 flights this month" or "what flights are available between these two dates") require efficient range access over dates. No single index type optimizes both patterns simultaneously, so a hybrid indexing strategy combining hashing and tree-based indexing is proposed.

1. Hash Index on Flight Number

Because point lookups by flight number are both extremely frequent and latency-sensitive (customers expect an almost instant response when searching for a specific flight), a hash index — ideally extendible hashing to accommodate the airline's growing route network — is built on Flight_Number. This provides average O(1) access, which is faster than the O(log n) a tree-based index would require for the same pure equality lookup.

2. Composite B+ Tree Index on (Flight_Number, Date)

For the very common combined query of a specific flight on a specific date or date range — for example, "AI202 departures between 20 and 27 December" — a composite B+ Tree index ordered first by Flight_Number and then by Date is built. The tree narrows the search to the target flight using the leading key, and because that flight's entries are then sorted chronologically at the leaf level, a linked-leaf range scan retrieves exactly the requested date window efficiently, without scanning unrelated flights.

3. Secondary B+ Tree Index on Date Alone

Airlines also need to answer queries that are date-driven but not tied to a single flight number, such as scheduling reports, capacity planning, or "list all flights departing tomorrow across the entire network." A standalone secondary B+ Tree index on Date supports these queries directly, independent of the flight-specific composite index.

Justification

A pure hash index, while excellent for point lookups, is fundamentally unsuitable for range queries, since hashing intentionally scatters keys and destroys any notion of order — a date-range query against a hash index would degenerate into scanning most of the structure. Conversely, relying solely on a B+ Tree on Flight_Number would handle point queries correctly but less efficiently than hashing, since tree traversal costs O(log n) rather than the O(1) that hashing offers. By combining a hash index for the highest-frequency point-access pattern with composite and secondary B+ Tree indexes for the range-based patterns, the system avoids compromising on either requirement, at the reasonable cost of maintaining more than one index structure — an acceptable trade-off given how central both speed and flexibility are to a booking-critical, revenue-generating system.

Conclusion

A hybrid indexing strategy — a hash index on Flight_Number for fast point queries, combined with a composite B+ Tree on (Flight_Number, Date) and a standalone B+ Tree on Date for range-based queries — together provide comprehensive, efficient support for the airline reservation system's dual access requirements.

Q7. For a university student database, design the file organization and secondary indexing to support fast queries by department and CGPA range.

A university's student information system is typically queried in two recurring ways: administrative staff frequently need to work with an entire department's student list at once (for batch processing such as attendance rosters, fee reminders, or departmental reports), while academic offices need to identify students within specific CGPA bands for purposes such as merit scholarships, academic probation review, or honors-list generation. Because department-based access is both extremely common and naturally groupable, it is a strong candidate for the physical (clustering) organization of the file, while CGPA, being a numeric range-query attribute cutting across departments, is better served by a secondary index layered on top.

1. Primary File Organization — Clustered Sequential File on Department

The student file is physically organized as a sequential file clustered by Department, meaning all students belonging to the same department are stored contiguously on disk. This is chosen as the primary (clustering) organization because department is a relatively low-cardinality, stable attribute — there are only a limited, fixed number of departments — and because department-scoped batch operations (fee processing, class-list generation, departmental audits) are among the most frequent administrative operations performed on the database. Clustering by department means these operations can be satisfied with a single contiguous block scan rather than scattered random I/O across the entire student file.

2. Secondary Index — B+ Tree on CGPA

Since the file is not physically sorted by CGPA, a non-clustering secondary B+ Tree index is built on the CGPA attribute. Because CGPA queries are inherently range-based (for example, "CGPA between 8.0 and 9.0"), a B+ Tree is the natural choice: its leaf nodes are linked together in sorted order, so once the starting CGPA value is located, the system can simply follow leaf pointers forward to retrieve every record in the requested range, without needing to scan the whole file or perform repeated tree traversals.

3. Supporting Combined Queries (Department and CGPA Together)

A very common real-world query combines both predicates at once, such as "list Computer Science students with CGPA above 8.5." This can be answered in two ways: first, by using the clustering organization to isolate the block range belonging to the target department and then applying the CGPA secondary index restricted to that subset of records; or second, and more efficiently for high query volume, by building a dedicated composite secondary B+ Tree index on (Department, CGPA), which allows the combined condition to be evaluated directly in a single index traversal rather than requiring a two-step filter-and-intersect process.

Justification

Department is chosen as the clustering key rather than CGPA or Student ID because department-wise operations occur with far greater administrative frequency, and grouping records physically by department minimizes disk-head movement and random I/O for the university's most routine batch workloads. CGPA, by contrast, is not suitable as a clustering key on its own, because doing so would scatter each department's students across the file, undermining the department-based batch operations that are the system's primary workload; it must instead be layered on as a secondary index.

Conclusion

A sequential file clustered by Department, combined with a secondary B+ Tree index on CGPA and, where combined query volume justifies it, a composite secondary index on (Department, CGPA), provides efficient support for both the university's routine department-scoped operations and its cross-departmental, CGPA-driven academic queries.

Q8. Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.

Disk block utilization is a critical, often underappreciated design consideration: a file organization that offers fast search or fast insertion frequently achieves this by deliberately leaving some space unused within each block, trading storage efficiency for operational efficiency. To analyze this trade-off concretely, consider a file of 100,000 records, each 50 bytes in size, stored using a 4KB (4096-byte) block size.

Baseline Calculation

The blocking factor — the number of records that fit into a single block — is calculated as the floor of the block ⌋ ⌉ 1,235 blocks. This figure serves as the baseline against which each file organization's actual space requirement is compared.

Heap File

A heap file imposes no ordering constraint and requires no reserved space for future insertions in sorted position, since new records are simply appended wherever space is available, most commonly at the end of the file. As a result, blocks can be packed essentially to capacity, giving a block utilization close to 100%, with the possible exception of the final, partially filled block. This gives a total requirement of approximately 1,235 blocks — the theoretical minimum. The cost of this excellent space efficiency is poor search performance, since locating any given record requires, on average, scanning half the file (approximately 617 block accesses for an average successful search, and the full 1,235 for an unsuccessful one).

Sorted File

A sorted file, in principle, requires no additional space purely to maintain sort order — the same 1,235 blocks would suffice if the file were rebuilt from scratch and never modified again. In practice, however, sorted files that must accommodate ongoing insertions typically reserve free space within each block (a fill factor, commonly around 70–80%) so that new records can be inserted into their correct sorted position without immediately triggering a full-file reorganization. Assuming a practical fill factor of about 75%, the effective space requirement rises to approximately $1{,}235 \div 0.75 \approx 1{,}647$ blocks. This additional ~33% storage overhead is the price paid to keep the sorted order economically maintainable under a stream of insertions, rather than needing to rewrite the entire file after every insert.

Hashed File

A hashed file organization deliberately under-fills its buckets — typically also to a 70–80% load factor — specifically to minimize the frequency of hash collisions and the resulting overflow chains, since collisions directly degrade the O(1) lookup performance that hashing is meant to provide. Using the same 75% target load factor, the base requirement is again approximately 1,647 blocks, but on top of this, hashed files must further reserve additional overflow blocks (commonly estimated at 10–15% extra) to absorb buckets that do collide despite the lowered load factor. This brings the total effective requirement to approximately 1,850 to 1,900 blocks — the largest of the three organizations.

Summary

Organization	Block Utilization	Approx. Total Blocks
Heap	~100%	1,235
Sorted (with fill factor)	~75–80%	~1,647
Hashed (with overflow)	~70–80% + overflow	~1,850–1,900

Conclusion

Heap files achieve the best possible block utilization because they impose no structural constraints on where records are placed, but this comes at the cost of the worst search performance of the three organizations. Sorted and hashed files both deliberately sacrifice some block utilization — reserving free space via a fill factor or load factor — specifically to keep the cost of future insertions and collisions manageable; hashed files require the most additional space overall because, beyond the load-factor reservation, they must also budget for overflow blocks to absorb unavoidable collisions.

Q9. A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.

At the scale of 500 million posts, a social media platform's indexing requirements are shaped by three fundamentally different query patterns that a single index cannot serve efficiently at once: retrieving an individual user's timeline in chronological order, searching for all posts carrying a specific hashtag regardless of who posted them, and answering global, platform-wide, time-bounded queries such as identifying trending content within the last hour. This necessitates a multi-index, specialized architecture rather than any single composite index.

1. Clustering Index — B+ Tree on (user_id, timestamp)

Posts are physically clustered first by user_id and then sorted by timestamp within each user's set of posts. This directly supports the platform's single most frequent read operation — loading a user's own timeline or profile feed, most recent posts first — as an efficient, contiguous range scan rather than a scattered lookup across the entire 500-million-post store.

2. Secondary Structure — Inverted Index on Hashtag

Hashtag search represents a many-to-many relationship: a single post may carry several hashtags, and a single hashtag may be attached to millions of posts. This access pattern is best served not by a traditional B+ Tree but by an inverted index, in which each hashtag maps to a posting list — an array or sorted list of post identifiers, typically maintained in timestamp order to directly support trending and recency-based hashtag queries. Given the scale of 500 million posts, this inverted index should itself be partitioned or sharded across multiple index servers, for example using consistent hashing over the hashtag key space, so that no single server becomes a bottleneck for popular hashtags.

3. Secondary Index — LSM-Tree on Timestamp

Global, cross-user, time-bounded queries — such as identifying all posts created within the last hour for trending-topic analysis — require an index on timestamp that is independent of user_id. Because social media platforms sustain an extremely high, continuous write rate (new posts arriving constantly), a Log-Structured Merge Tree (LSM-Tree) is preferred here over a traditional B+ Tree, since LSM-Trees are specifically optimized for high-throughput sequential writes, buffering incoming writes in memory and periodically flushing and merging them to disk, whereas a B+ Tree's in-place random-access updates would become a write bottleneck at this insert volume.

Justification

No single composite index can efficiently serve all three of these access patterns simultaneously at a scale of 500 million records, because the patterns require fundamentally different physical orderings of the same underlying data: by-user-then-time for timelines, by-hashtag for topic search, and by-time-alone for global trend analysis. This is why large-scale, real-world social platforms (as reflected, for instance, in the publicly described architectures of systems like Twitter) separate timeline storage from search/indexing infrastructure rather than attempting to force one physical structure to serve every query type. Additionally, because posting is a write-heavy workload, the choice of LSM-Trees for the write-hot timestamp index, versus a more traditional clustering B+ Tree for the comparatively more stable per-user timeline, reflects a deliberate matching of index technology to the specific read/write characteristics of each access pattern.

Conclusion

A composite indexing strategy consisting of (a) a clustering B+ Tree on (user_id, timestamp) for user timelines, (b) a sharded inverted index on hashtag for topic-based search, and (c) an LSM-Tree on timestamp.

Q10. Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.

A supermarket billing system is characterized above all by an extremely high, continuous insertion rate — roughly 1 million new transaction records generated every single day at checkout counters across the store network — combined with comparatively infrequent, but latency-sensitive, point lookups, such as reprinting a lost receipt or processing a refund or void against a specific transaction. Range-based reporting (daily sales summaries, tax reporting, inventory reconciliation) is typically important but is usually delegated to a separate analytics or data-warehouse system that is refreshed periodically, rather than being served directly from the live, high-throughput transactional store. This workload profile makes a direct comparison of insert, delete, and search performance across Heap, Sorted, and Hashed file organizations particularly instructive.

Performance Comparison

Operation	Heap File	Sorted File	Hashed File
Insert	O(1) — simple append to end of file	O(n) — must locate correct sorted position and shift/reorganize	O(1) — direct bucket computed via hash function
Delete	O(n) — must search then mark/remove record	O(n) — search plus shifting to preserve sort order	O(1) — locate bucket directly and remove
Search (by key)	O(n) — linear scan required	$O(\log_2 n)$ — binary search possible due to sort order	O(1) average — direct bucket access
Range query	O(n) — no exploitable ordering	$O(\log_2 n) + O(k)$ — efficient, exploits sort order	Poor — hashing destroys order, approaches O(n)

Insert-Heavy Workload Analysis

Given the volume of roughly 1 million inserts per day, the cost of the insert operation dominates the overall system load far more than any other single factor. Both the Heap File and the Hashed File offer O(1) insertion, making them immediately far more suitable than the Sorted File, whose O(n) insert cost — driven by the need to locate the correct sorted position and shift subsequent records, or periodically reorganize the file — would be prohibitively expensive if applied 1 million times per day; at this volume, a sorted file would quickly become a severe performance bottleneck and would likely require the system to fall behind during peak shopping periods.

Search Performance for Receipts and Refunds

Although searches are comparatively less frequent than inserts in this workload, they are highly latency-sensitive from a customer-service perspective — a cashier processing a refund or reprinting a receipt needs the specific transaction record essentially instantly. Here the Hashed File decisively outperforms the alternatives, offering O(1) average-case lookup by Transaction_ID compared to the Heap File's O(n) linear scan, which would be unacceptably slow for a file accumulating a million new records daily. The Sorted File's $O(\log_2 n)$ binary search is respectable but still inferior to hashing for pure equality lookups, and it comes bundled with the unacceptable O(n) insert cost already discussed.

Delete Performance for Voided Transactions

Voided or corrected transactions require deletion (or logical marking) of specific records. The Hashed File again performs best here, at O(1), by directly locating the relevant bucket. The Sorted File incurs the same reorganization cost on delete as it does on insert, since removing a record and preserving sort order typically requires shifting subsequent records, while the Heap File requires an O(n) search before the record marked for deletion can even be located.

The Range Query Trade-off

The one area where the Hashed File is clearly weaker is range querying, since hashing deliberately scatters keys and provides no exploitable ordering — a request such as "all transactions between 9 a.m. and 12 p.m." would perform poorly against a pure hash structure, approaching a near-complete scan. In an idealized single-structure design this would be a serious drawback; however, in the specific context of this supermarket billing system, range-based reporting and analytics needs are typically handled by a separate, periodically refreshed reporting or data-warehouse layer rather than by direct range queries against the live transactional hash store, which makes this weakness an acceptable and manageable trade-off rather than a disqualifying one.

Conclusion

For a supermarket billing system defined by very high daily insert volume together with frequent, latency-sensitive point lookups for receipts, refunds, and voids, the Hashed File organization is the best overall choice. It delivers O(1) performance on the three operations that matter most in this workload — insert, delete, and search — while its comparative weakness at range queries is appropriately offloaded to a separate downstream reporting system rather than being solved within the live transactional store itself.